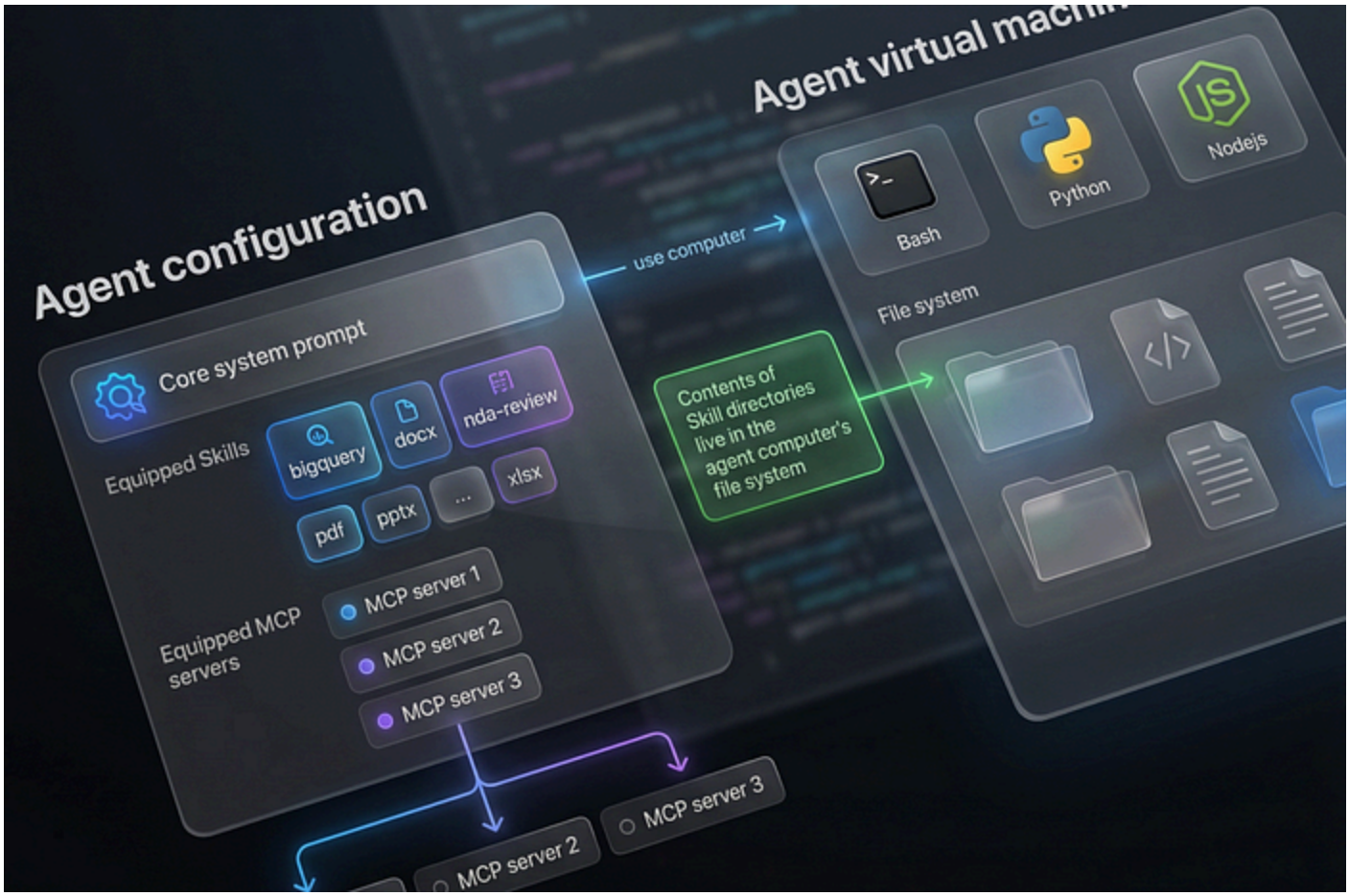
Milestone: 20GB Reached

We’ve reached 20GB of stored data — thank you for helping us grow!

Patreon Ko-fi Liberapay
Close
Freedium

- ko-fi
- librepay
- patreon

< Go to the original



You Can Copy Claude Skills to Any Agent: Anthropic SKILL System

Your agent is almost useful but every time you start a new task you're re-teaching it the same routine.

Agent Native Follow
~5 min read · February 1, 2026 (Updated: February 1, 2026) · Free: No

- "When you edit a PDF, do X."
- "When you touch the DB schema, follow Y."
- "When you open a PR, follow Z."

And then the chat gets long, context gets polluted, the agent starts hallucinating steps you swear you already clarified, and you end up doing the annoying parts yourself anyway.

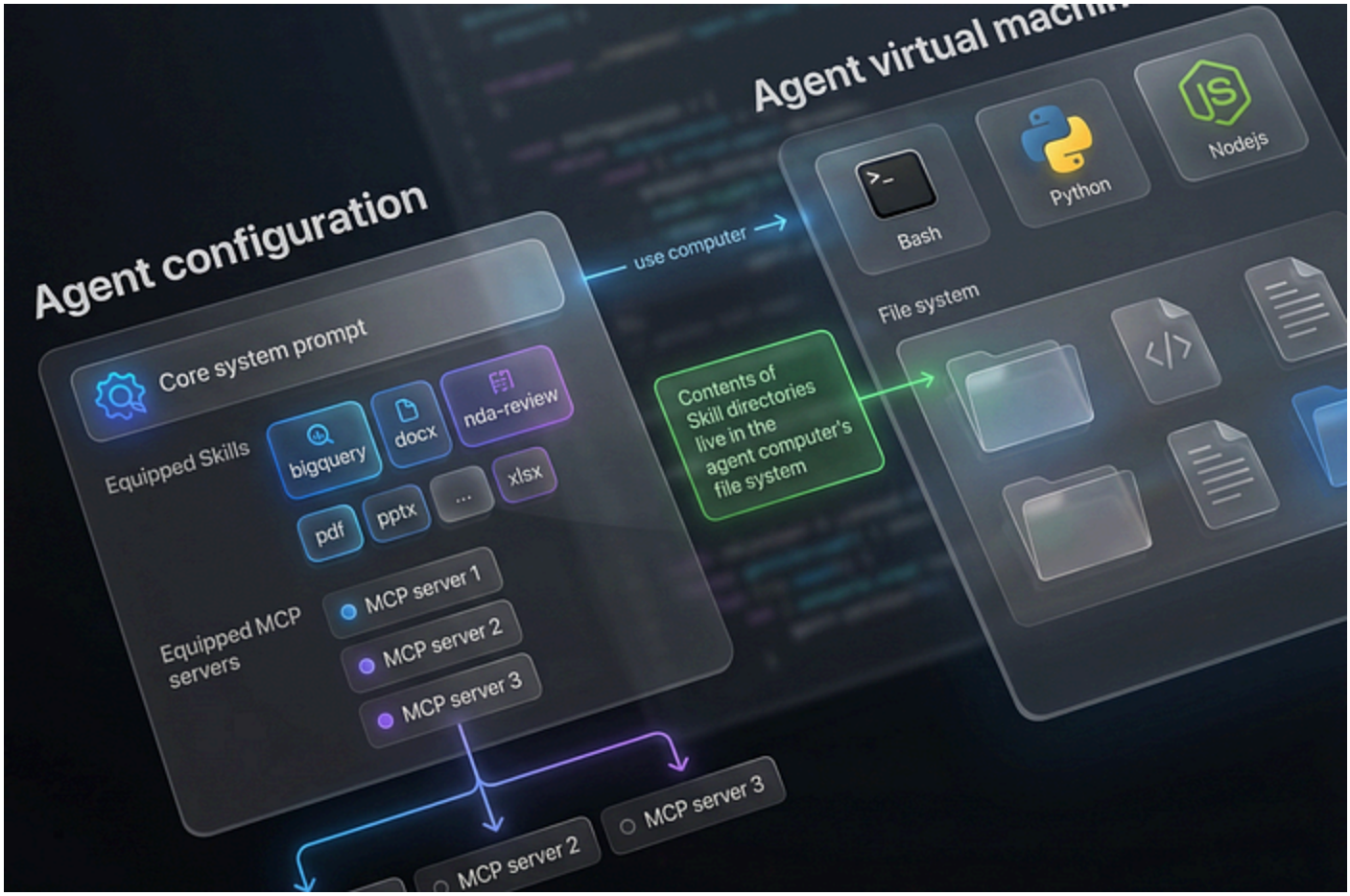
Better way is to **standardize skills as files (`SKILL.md`) and make them portable across agents**, using the same format popularized by Anthropic's skills system.

The key idea is **making instructions composable, versionable, and loadable on demand**.

This is basically that muscle memory stored as plain files, installed like dependencies, and injected only when needed.

> If you haven't received it yet, Issue #1 is live for [agentic SaaS patterns winning in 2026](), enjoy the issue.

Let me walk you through that.



Progressive disclosure for agent context

Most agent setups fail because **they treat all instructions as always-relevant**.

To improve agent performance, you should lean into a cleaner mental model:

1. **Metadata is always in context** (name + description).
2. **Full skill instructions load only when relevant**
3. **Heavy docs/scripts/assets don't automatically flood context**

This is **progressive disclosure where** the agent sees just enough to know what exists, and you only load the "real" instructions at the moment they matter.

What OpenSkills actually does

OpenSkills is a CLI that:

- installs skills from a repo, a local path, or a git URL
- writes a skills index into `AGENTS.md` (in the exact `<available_skills>` XML shape expected by the skills system)
- lets agents "load" a skill by printing it to stdout (`read` command)

From the developer perspective, it's basically:

- **`install`** → get skills into the right folder
- **`sync`** → update your `AGENTS.md` so agents can discover them
- **`read`** → load a skill into the agent context on demand
- **`update`** → refresh installed skills from recorded sources
- **`manage/remove`** → clean up without manual file spelunking

If you run:

```
npx openskills install anthropics/skills
npx openskills sync
```

you've done two important things:

1. You installed skill folders into a standard location (project-local by default).
2. You generated/updated a `AGENTS.md` section that lists skills and tells the agent how to load them.

That second part is the secret sauce.

OpenSkills isn't trying to convince every agent vendor to implement a new plugin ecosystem.

It instead says:

> "If your agent can read a markdown file and run shell commands, it can use skills."

That's why it's "universal."

Here's an example structure:

```
my-skill/
├── SKILL.md
├── references/
│   ├── overview.md
│   └── api.md
├── scripts/
│   └── check.sh
└── assets/
    └── template.md
```

read: the on-demand skill loader your agent can actually use

This is the part that makes the system feel alive.

You can:

- print the skill instructions to stdout
- make the agent ingests that text into context

Example:

```
npx openskills read pdf
```

And if you want more than one:

```
npx openskills read foo,bar
```

This makes skills "portable" in a way most agent plugin systems aren't:

- no daemon
- no server
- no tool schema negotiation
- just files and shell output

SKILL.md for authoring skills

The repo includes a "skill format reference," and it's worth following because it forces you into good habits.

Every `SKILL.md` starts with:

```
---
name: skill-name
description: When to use this skill in 1-2 sentences.
---
```

and descriptions are what get indexed in `<available_skills>`.

If you write vague descriptions, your agent will "discover" the wrong skill or never find the right one.

A good description reads like an IDE tooltip:

> "Automates X workflow, including A/B/C, and provides guardrails for D."

You should format your statements as

- "Do X, then Y"
- and avoid "You should..." / "If you need..." fluff

This pushes you toward writing skills like a runbook.

And it makes skills easier to test mentally, if you can't imagine the agent following it step by step, the skill is probably too vague.

It's also best practice to keep SKILL.md under ~5,000 words.

This is not a hard technical limit, but you should keep the "core" small, move details into references and keep scripts executable.

Installing skills from anywhere

You can also intall skills from multiple sources:

- "repo shorthand" like `owner/repo` (it converts to a GitHub URL)
- `owner/repo/path/to/skill` (install just one skill)
- full git URLs (SSH, HTTPS, `.git`)
- local directories (`./my-skill`, `~/skills/foo`)

If a skill already exists, OpenSkills prompts to overwrite unless you're in `--yes` mode, where it auto-overwrites and logs what it did.

The `update` command is one of those features that sounds minor but important.

```
npx openskills update
```

Copy

OpenSkills can refresh skills **from their recorded source metadata** (repo URL + subpath, or local path).

If your application depends on skills for safety guardrails (CI rules, release steps, security checks), you should also treat skills like code:

- pin skill repos (or vendor them)
- review changes via PR
- sync `AGENTS.md` as part of the merge

SKILL.md template that tends to work

This is not a literal copy-paste, but the structure that keeps skills "agent-friendly":

1. **When to use** (1 paragraph)
2. **Inputs required** (what the agent should ask for)
3. **Steps** (numbered, deterministic)
4. **Validation** (how to confirm success)
5. **Escalation** (when to stop and ask a human)
6. **References** (links to references/)
7. **Scripts** (what scripts exist, what they do)

The key is skills should reduce ambiguity.

For example, if you want a skill to be "discoverable," write the description like a search query

Instead of:

- "Helps with Git"

Do:

- "Standardizes Git workflow: branch naming, commits, PR checklist, and safe rebases."

Now your agent can match user intent to skill metadata.

Why CLI instead of MCP?

There is a sharp distinction:

- MCP is for **dynamic tools**
- skills are **static instructions + resources**

For skills particularly:

- no server required
- no runtime coupling exists
- and they work across agents without native MCP support

MCP is great when you need a tool call contract but skills are great when you need **behavioral consistency**.

A lot of "AI dev productivity" content is secretly about squeezing more out of one conversation.

But you can build more durable workflows by continuously curating and recalibrating your application's skillset.

Feel free to dive deeper into source code.

Bonus Articles

Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you...

medium.com

Think Your Emails Are Safe? OpenClaw (Clawdbot) Proves Otherwise

More than 1800+ instances of OpenClaw on the public internet, over half were running with an unauthenticated gateway...

medium.com

Fully Local Agentic Coding on localhost: Claude Code, Codex, llama.cpp, Ollama and Docker MR

This is the setup I wish I had the first time I tried to run Claude Code and Codex-style workflows against local...

medium.com

Kimi K2.5 + Agent Swarms Beat US AI Labs: Open Source vs OpenAI, Anthropic & Google

Kimi K2.5 is the most powerful open source model, ahead of GLM-4.7, DeepSeek V3.2 and Gemini 3 Pro, with only OpenAI...

medium.com

Qwen3-Max-Thinking Outperforms Claude Opus 4.5 and Gpt-5.2: The Panic Is Real

Alibaba's Qwen3 family of models has become a major alternative in the LLM ecosystem.

medium.com

See you next time around.
#anthropic-claude#claude-code#generative-ai-tools#ai-agent#ai-coding

Reporting a Problem

Sometimes we have problems displaying some Medium posts.

If you have a problem that some images aren't loading - try using VPN. Probably you have problem with access to Medium CDN (or fucking Cloudflare's bot detection algorithms are blocking you).

Problem Description Describe your problem here...

Submit Cancel